

Análise semântica

Constantino Martins
Luiz Faria

O compilador até à análise semântica

- Análise léxica
 - ◆ Separa o texto de entrada numa sequência de *tokens*
 - ◆ Detecta textos de entrada com *tokens* ilegais
- Análise sintáctica
 - ◆ Aplica regras gramaticais
 - ◆ Constrói (por vezes implicitamente) uma árvore de *parse* com os *tokens* de entrada
 - ◆ Detecta textos com sequências ilegais de *tokens* (erros sintácticos)
- Cada uma destas fases detecta certos erros de programação mas não todos
- A última fase envolvida na análise é a análise semântica
 - ◆ Detecta todos os restantes erros do texto de entrada (excepto os erros de concepção do programa e de execução)

Existem situações em que não é possível ao compilador efectuar alguns tipos de verificação. Por exemplo:

tabela: array[0..255] of char;

i: integer;

Não é possível garantir durante a compilação que **tabela[i]** é uma expressão válida

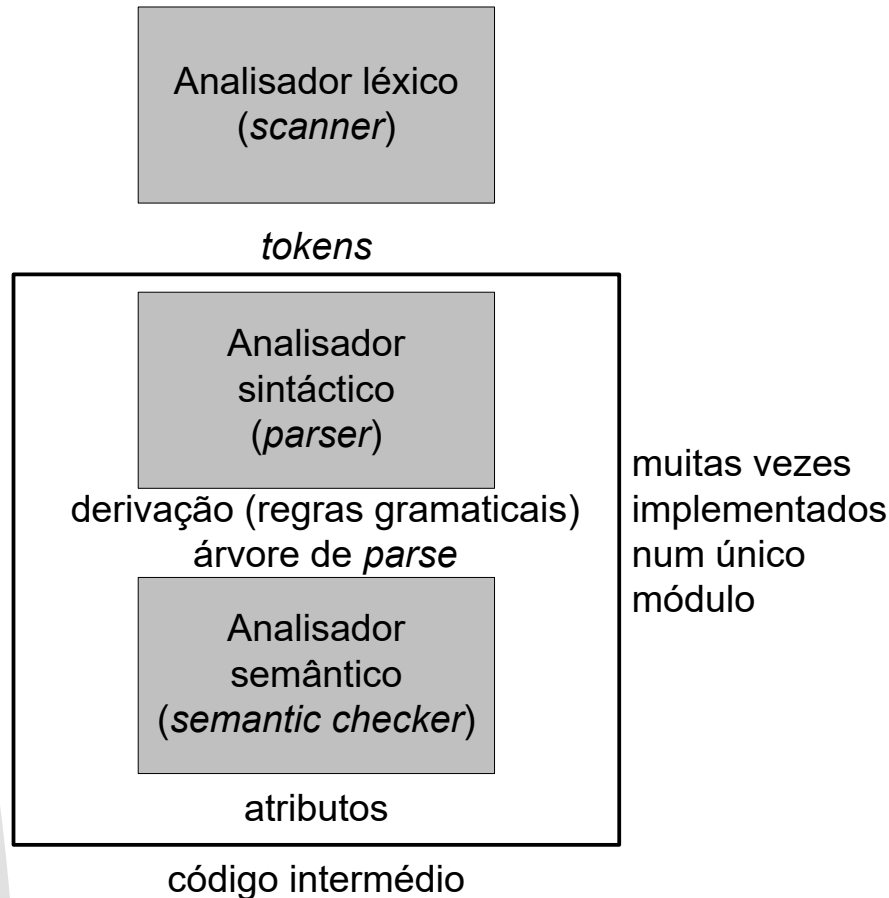
tabela[i] (com i=300)

NÃO é erro de compilação

é erro de execução

pois **i** pode vir a tomar valores fora do intervalo [0..255]

Fases do compilador (em revista)



Funções do analisador semântico

```
int x;  
x = "ola";
```

- Léxica?
- → OK
- Sintática?
- → OK
- Semântica?
- ERRO

Funções do analisador semântico

- Executar uma grande quantidade de verificações
 - ◆ todos os identificadores foram declarados
 - ◆ não existência de declarações múltiplas e incompatíveis para um mesmo identificador
 - ◆ determinar os tipos de expressões e variáveis e a sua compatibilidade quando aparecem numa mesma expressão
 - ◆ relações de herança (em linguagens OO)
 - ◆ boa utilização de palavras e identificadores reservados
 - ◆ não existência de métodos com definição múltipla numa classe (em linguagens OO)
- Os requisitos exactos dependem da linguagem que se está a analisar
- Praticamente todas as linguagens requerem as três primeiras verificações (excepto as linguagens que não requerem a declaração de identificadores)

identificador é o nome que damos a variáveis, funções ou classes.....

Implementação da análise semântica

- A análise semântica é implementada utilizando extensões das gramáticas livres de contexto utilizadas pelos analisadores sintácticos
- Essas extensões designam-se por definições dirigidas pela sintaxe (*syntax-directed definitions*)
- Estas definições são compostas por dois tipos de extensões das gramáticas independentes do contexto (GIC)
 - ◆ Atributos
 - ◆ Regras ou acções semânticas

Implementação da análise semântica

- Atributos - valores associados a cada símbolo (terminais e não-terminais) da gramática
 - ◆ Os atributos podem representar uma multiplicidade de propriedades dos símbolos gramaticais, tais como:
 - ★ Tipos das variáveis
 - ★ Valores de constantes e expressões
 - ★ Endereços das variáveis
 - ★ Endereços dos procedimentos e funções
- Regras ou acções semânticas - cálculos ou outras acções executadas quando se aplica uma regra gramatical (produção) na análise sintáctica
 - ◆ As acções são executadas quando o compilador reconhece uma regra

Atributos (informação)

- tipo
- valor
- nome

....

Ações (o que fazemos)

- verificar
- calcular
- detetar erros

....

Atributos e acções

- Considere-se uma produção de uma GIC da forma:

$$Y \rightarrow X_1 X_2 \dots X_n$$

- ◆ Os atributos são valores associados aos símbolos da gramática (nós da árvore de *parse*). Por exemplo poderão existir os atributos *Y.type*, *X1.val*, *X2.type*, etc, associados aos símbolos da regra anterior. Cada símbolo pode ter vários atributos
- ◆ **Os símbolos terminais obtêm os seus atributos do analisador léxico (valor de uma constante, nome de uma variável, etc.)**
- ◆ Cada símbolo pode ter informação associada:
 - ★ valor (ex: 5)
 - ★ tipo (int, float)
 - ★ Nome
 - ★ “São propriedades dos símbolos.”
- ◆ As acções são pedaços de código associados à regra

Exemplos

Exemplo:

Considere-se a seguinte regra de uma gramática: $N \rightarrow N \text{ digit}$ em que o não-terminal N e o terminal digit possuem um atributo chamado val .

Esta regra e a respectiva acção semântica poderia ser escrita como:

$N_1 \rightarrow N_2 \text{ digit} \{N_1.\text{val} = N_2.\text{val} \times 10 + \text{digit}.\text{dval};\}$

Os índices distinguem apenas as várias instâncias de N dentro da regra

Outro exemplo : Duas regras de uma gramática e respectivas acções embebidas

$R1 \rightarrow + \{ \text{print}('+'); \} T R2$

$T \rightarrow \text{num} \{ \text{print}(\text{num}.\text{val}); \}$

Exemplo 1

Cálculo de uma expressão

3 + 5

Regras da gramática simplificado:

$E \rightarrow E + T \{ E.val = E1.val + T.val \}$

$E \rightarrow T \{ E.val = T.val \}$

$T \rightarrow \text{numero} \{ T.val = \text{numero.val} \}$

Cada símbolo tem um atributo **val** (valor):

- 3.val = 3
- 5.val = 5

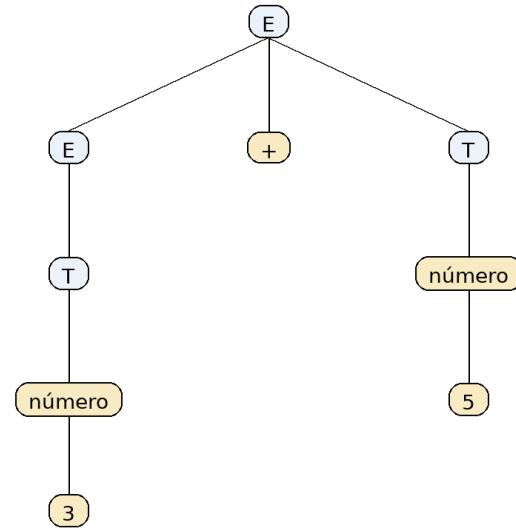
Ação Semântica

$E.val = E1.val + T.val$

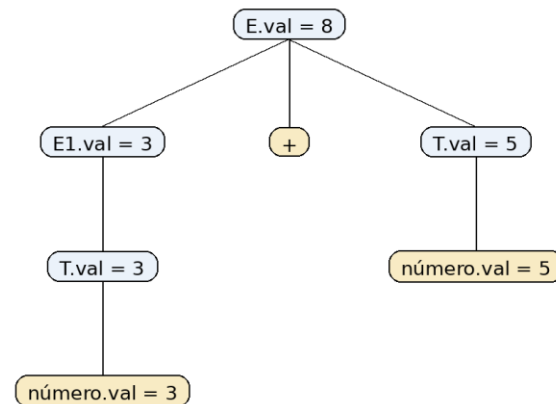
$E.val = 3 + 5 = 8$

A gramática reconhece '3 + 5',
mas os atributos permitem calcular
o resultado.

Árvore de Derivação



Árvore Semântica



Exemplo 2

```
int a;  
a = 3 + 5;
```

Cada símbolo pode ter:

- type → tipo (int, float, ...)
- val → valor (quando aplicável)
- name → nome da variável

Expr1 é só uma etiqueta explicativa
Por exemplo, numa implementação
real seria usado:
variáveis como left e right, ou
posições como \$1, \$3, \$\$
o símbolo da gramática continua a
ser só Expr

Extrato das regras e ações semânticas

```
Decl → int id  
{ id.type = integer }
```

```
Expr → Term  
{  
  Expr.type = Term.type  
  Expr.val = Term.val  
}
```

```
Term → número  
{  
  Term.type = integer  
  Term.val = numero.val  
}
```

```
Expr → Expr1 + Term  
{  
  if (Expr1.type != Term.type)  
    error("tipos incompatíveis")  
  else {  
    Expr.type = Expr1.type  
    Expr.val = Expr1.val + Term.val  
  }  
}
```

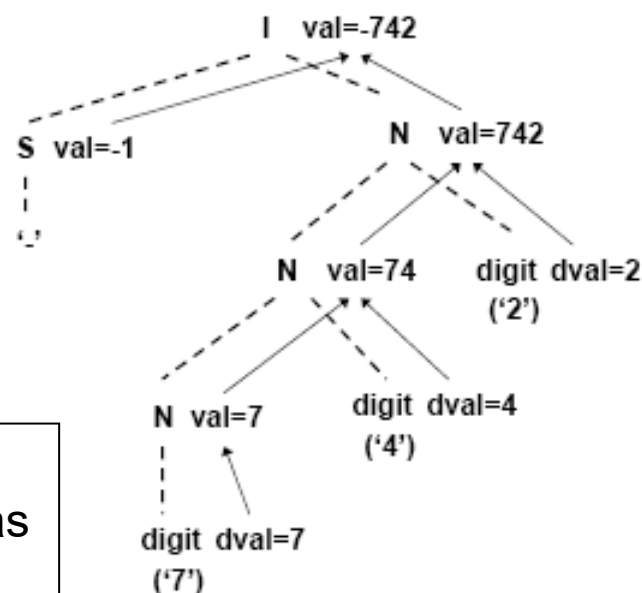
```
Assign → id = Expr  
{  
  if (id.type != Expr.type)  
    error("tipos incompatíveis na atribuição")  
}
```

Exemplo 3

Considere-se a seguinte gramática para números inteiros em que os *tokens* são os dígitos ('0' a '9') e os sinais '+' e '-'. O objectivo é a determinação do valor de cada inteiro. Para isso associa-se aos não-terminais I, S e N um atributo inteiro val e pressupõe-se que o analisador léxico retorna para o *token* 'digit' um atributo dval (digit value) entre 0 e 9.

Gramática	Acções semânticas
$I \rightarrow S N$	$I.val = S.val \times N.val$
$S \rightarrow +$	$S.val = 1$
$S \rightarrow -$	$S.val = -1$
$S \rightarrow \epsilon$	$S.val = 1$
$N_1 \rightarrow N_2 \text{ digit}$	$N_1.val = N_2.val \times 10 + \text{digit}.dval$
$N \rightarrow \text{digit}$	$N.val = \text{digit}.dval$

Seguindo esta gramática, uma árvore de *parse* para o texto -742 seria:

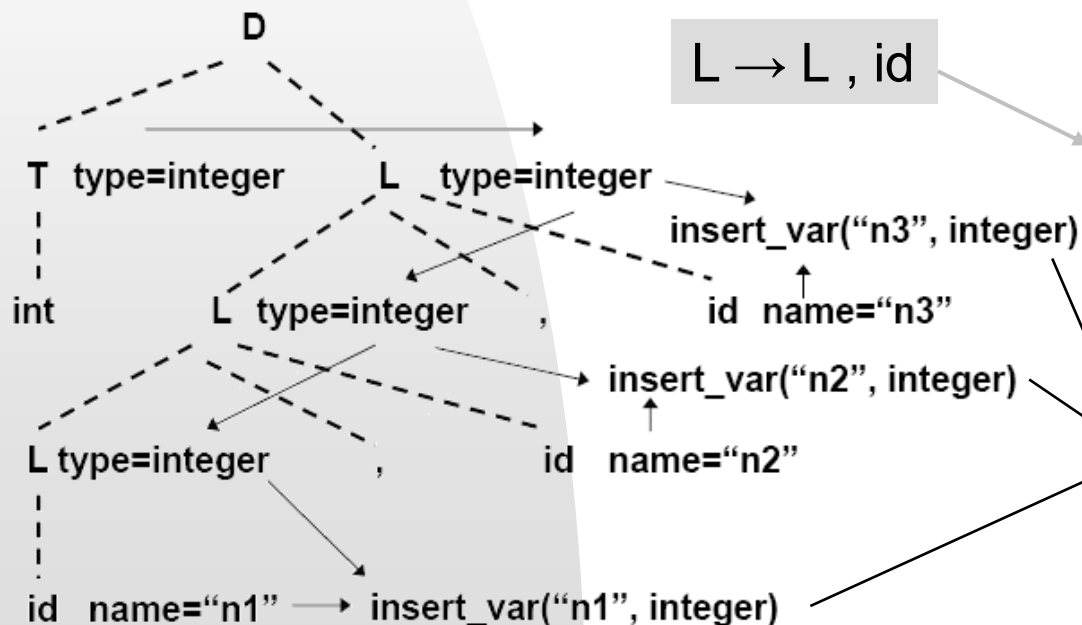


As acções semânticas estabelecem dependências entre os atributos, indicadas pelas setas, nesta árvore de *parse*

Exemplo 4

Seja agora a seguinte gramática para declarações de variáveis (numa linguagem tipo C). O objectivo é adicionar a uma estrutura de dados externa – a tabela de símbolos – os nomes das variáveis declaradas, juntamente com o seu tipo. Para isso associa-se aos não-terminais T e L um atributo type que pode ter os valores real e integer. Pressupõe-se que o analisador léxico retorna o atributo name para os tokens id. A função `insert_var(name, type)` insere uma nova variável na tabela de símbolos.

Considere-se o seguinte texto e a respectiva Árvore de *parse* e execução das acções:
“int n1, n2, n3”



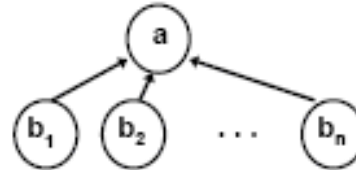
Gramática	Acções semânticas
$D \rightarrow T L$	$L.type = T.type$
$T \rightarrow int$	$T.type = integer$
$T \rightarrow float$	$T.type = real$
$L_1 \rightarrow L_2, id$	$L_2.type = L_1.type$ <code>insert_var(id.name, L1.type)</code>
$L \rightarrow id$	<code>insert_var(id.name, L.type)</code>

Tabela de símbolos:

n1	integer
n2	integer
n3	integer

Atributos sintetizados e herdados

- As acções semânticas definem dependências entre os atributos
 - ◆ Se, por exemplo, existir uma acção do tipo $a = f(b_1, b_2, \dots, b_n)$, onde a e b_1 a b_n são atributos associados aos símbolos de uma regra, diz-se que a depende dos atributos b_1 a b_n
 - ◆ Essa dependência pode exprimir-se através de um grafo (de dependências) onde os atributos são nós, e há ramos dirigidos dos atributos usados no cálculo para os atributos dependentes:



- ◆ Se na totalidade do conjunto de acções de uma gramática **apenas houver atributos que dependam dos atributos dos filhos** numa árvore de *parse*, diz-se que essa gramática (estendida com atributos e acções) é ***S-attributed*** e esses atributos dizem-se **sintetizados** (exemplo 1 atrás)
- ◆ Se houver dependências entre atributos de símbolos do **lado direito das regras (filhos)**, ou estes dependerem de atributos do não-terminal do lado esquerdo (pai), estes atributos dizem-se **herdados**
- ◆ Se, para os atributos herdados, estes dependerem **apenas de atributos à sua esquerda ou do pai**, diz-se que essa gramática é ***L-attributed*** (exemplo 4 atrás)

Cálculo dos atributos

- Os atributos são calculados nas acções semânticas. As acções de cálculo só podem ser executadas quando todos os atributos de que dependem já estiverem disponíveis
- As acções de todas as gramáticas *S-attributed* e de muitas outras que sejam *L-attributed* podem ser executadas durante a análise sintáctica pelo *parser* e sem necessidade da construção explícita da árvore de *parse*
- No caso geral poderá ser necessário construir explicitamente a árvore de *parse* e o grafo de dependências
 - ◆ Cálculo dos atributos no caso geral:
 - ★ Construir a árvore de *parse*
 - ★ Construir o grafo de dependências
 - ★ Fazer uma ordenação topológica do grafo de dependências
 - ★ Seguir essa ordenação
 - ◆ Para que seja possível efectuar a ordenação topológica o grafo de dependências terá que ser acíclico

Exemplo

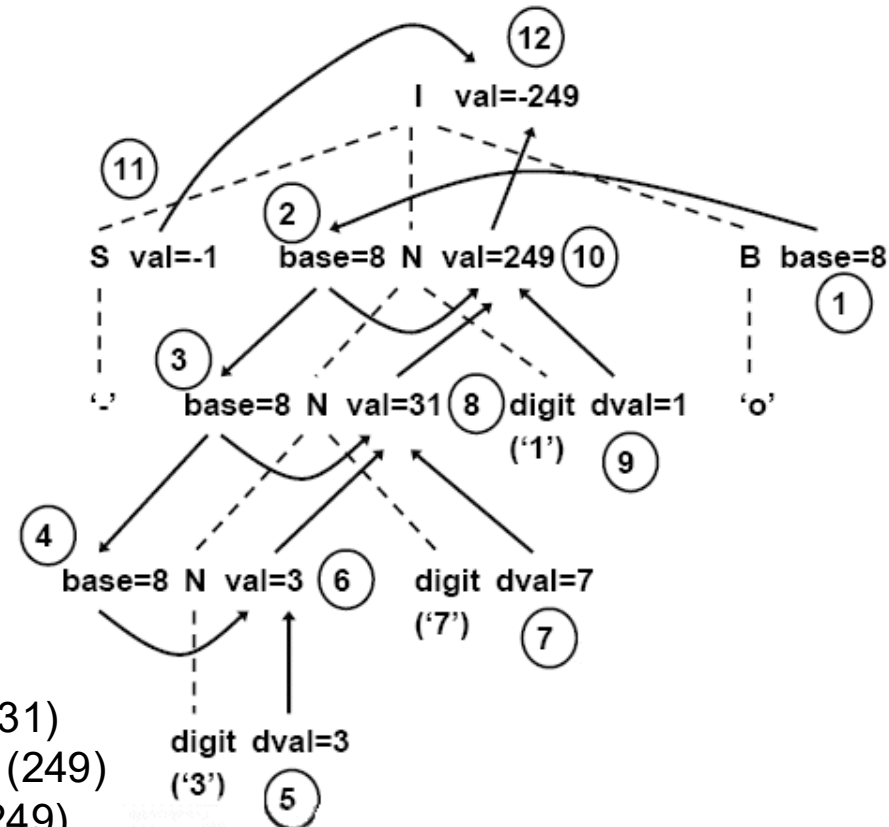
Considere-se novamente a gramática do exemplo 4 da página 13, mas agora estendida de modo a indicar a base do número inteiro (octal ou decimal):

Gramática	Acções semânticas
$I \rightarrow S N B$	$I.val = S.val \times N.val$ $N.base = B.base$
$B \rightarrow o$	$B.base = 8$
$B \rightarrow d$	$B.base = 10$
$S \rightarrow +$	$S.val = 1$
$S \rightarrow -$	$S.val = -1$
$S \rightarrow \epsilon$	$S.val = 1$
$N_1 \rightarrow N_2 \text{ digit}$	$N_1.val = N_2.val \times N_1.base + \text{digit}.dval$ $N_2.base = N_1.base$ if ($N_1.base == 8 \ \&\& \ \text{digit}.dval \geq 8$) $N_1.val = 0$; error()
$N \rightarrow \text{digit}$	$N.val = \text{digit}.dval$ if ($N.base == 8 \ \&\& \ \text{digit}.val \geq 8$) $N.val = 0$; error()

Pode ver-se uma ordenação topológica do grafo de dependências. Se chamarmos a_k ao atributo do nó k , temos os seguintes cálculos:

$$\begin{aligned}
 a_2 &= a_1 (8) & a_8 &= a_6 \times a_3 + a_7 (31) \\
 a_3 &= a_2 (8) & a_{10} &= a_8 \times a_2 + a_9 (249) \\
 a_4 &= a_3 (8) & a_{12} &= a_{11} \times a_{10} (-249) \\
 a_6 &= a_5 (3)
 \end{aligned}$$

Vejamos qual é a árvore de *parse* e o grafo de dependências para o texto: "- 371 o"



Accções e atributos durante a análise sintáctica *top-down*

- Praticamente todas as gramáticas *S-attributed* ou *L-attributed* são de fácil avaliação durante a análise sintáctica num *parser* de descida recursiva (*top-down*)
- A técnica a usar é a seguinte:
 - ◆ Nestes *parsers* há um procedimento para expandir cada símbolo gramatical não-terminal
 - ◆ Estes procedimentos são substituídos por funções onde:
 - ★ os atributos herdados do símbolo a expandir são passados como argumentos da função
 - ★ os atributos sintetizados são calculados na função e retornados por esta
 - ◆ Como os atributos herdados numa gramática *L-attributed* dependem apenas de atributos à esquerda numa regra, quando se chama a função de expansão já deverão estar disponíveis para serem passados como parâmetros
 - ◆ Os atributos sintetizados dependem apenas dos filhos, que vão retornando os respectivos atributos, permitindo o cálculo dos atributos actuais e o seu retorno a quem chamou a função

Cálculo de atributos sintetizados em *parsers bottom-up*

- Os parsers *bottom-up* (SLR/LALR/LR) mantêm uma *stack* onde vão sendo armazenados os terminais e não-terminais. Quando os n símbolos do topo da *stack* coincidem com a parte direita de alguma regra gramatical, podem ser substituídos pelo símbolo da parte esquerda (efectua-se uma redução)
- É na altura das reduções que se devem executar as acções semânticas
- Para isso a *stack* do *parser* é estendida por forma a incorporar os atributos associados a cada símbolo; (cada elemento da *stack* pode ser uma estrutura com vários campos)
- Os atributos sintetizados dependem apenas dos atributos dos filhos, numa árvore de *parse*
- Os filhos, na aplicação de uma regra sintáctica, estão no topo da *stack* do *parser*

Exemplo: Regra $E1 \rightarrow E2 + T$ $\{E1.val = E2.val + T.val\}$

Símb.	Val		Símb.	Val
T	10	→		
+				
E	35		E	45

Exemplo

Cálculo de expressões de dígitos:

Gramática	Acções semânticas
$L \rightarrow E$	<code>print(val[top])</code>
$E_1 \rightarrow E_2 + T$	<code>val[top-2]=val[top-2] + val[top]</code>
$E \rightarrow T$	
$T_1 \rightarrow T_2 * F$	<code>val[top-2]=val[top-2] * val[top]</code>
$T \rightarrow F$	
$F \rightarrow (E)$	<code>val[top-2]=val[top-1]</code>
$F \rightarrow \text{digito}$	

$\text{top} \rightarrow$ operando direito
 $\text{top-1} \rightarrow$ operador
 $\text{top-2} \rightarrow$ operando esquerdo
 $\text{val}[\text{top-2}] = \text{val}[\text{top-2}] \text{ op } \text{val}[\text{top}]$
 $3+5$
 $[3 , + , 5] \rightarrow 8$

Configuração do *parser* durante a análise de:
 $3 * 5 + 4 \$$

Entrada	Símbolo	Val	Regra
3*5+4\$			
*5+4\$	3	3	
*5+4\$	F	3	$F \rightarrow \text{digito}$
*5+4\$	T	3	$T \rightarrow F$
5+4\$	T *	3 -	
+4\$	T * 5	3 - 5	
+4\$	T * F	3 - 5	$F \rightarrow \text{digito}$
+4\$	T	15	$T \rightarrow T * F$
+4\$	E	15	$E \rightarrow T$
4\$	E +	15 -	
\$	E + 4	15 - 4	
\$	E + F	15 - 4	$F \rightarrow \text{digito}$
\$	E + T	15 - 4	$T \rightarrow F$
\$	E	19	$E \rightarrow E + T$
\$	L	19	$L \rightarrow E$ (<code>print(19)</code>)

Implementação de acções a meio em *parsers bottom-up*

- Nos esquemas de tradução aparecem acções semânticas no meio das produções
- Num *parser bottom-up* só é possível executar as acções semânticas quando de uma redução de uma regra gramatical
- Como se implementam então as execuções destas acções semânticas ?
- Com a introdução de novos não-terminais marcadores

Gramática e semântica para a escrita pós fixa de expressões (tradução de um expressão em notação infixa para notação pós fixa):

$$\begin{aligned} E &\rightarrow T R \\ R &\rightarrow + T \{\text{print}(' + ')\} R \mid \\ &\quad - T \{\text{print}(' - ')\} R \mid \\ &\quad \epsilon \\ T &\rightarrow \text{num} \{\text{print}(\text{num.val})\} \end{aligned}$$
$$\begin{aligned} E &\rightarrow T R \\ R &\rightarrow + T M R \mid \\ &\quad - T N R \mid \\ &\quad \epsilon \\ T &\rightarrow \text{num} \{\text{print}(\text{num.val})\} \\ M &\rightarrow \epsilon \{\text{print}(' + ')\} \\ N &\rightarrow \epsilon \{\text{print}(' - ')\} \end{aligned}$$

Os novos não-terminais M e N servem apenas para a execução das acções semânticas que aparecem no seu lugar

Atributos herdados em *parsers bottom-up*

- Certos atributos herdados podem facilmente ser implementados num *parser bottom-up*
 - ◆ Num *parser bottom-up* uma regra do tipo $A \rightarrow XY$ é aplicada quando na *stack* do *parser* já existem os símbolos X e Y (este no topo); eles são então substituídos pelo símbolo A
 - ◆ No entanto quando o símbolo Y é construído (através de reduções na sua sub-árvore), já se encontram disponíveis quaisquer atributos sintetizados de X (na *stack* do *parser*)
 - ◆ Estes atributos podem então ser herdados por um ou mais atributos de Y : por exemplo pela regra $Y.i = X.s$, em que i é um atributo herdado de Y e s um atributo sintetizado de X

Exemplo

Consideremos novamente a gramática para declarações:

Gramática	Acções semânticas
$D \rightarrow T$ L	$L.type = T.type$
$T \rightarrow int$	$T.type = integer$
$T \rightarrow float$	$T.type = real$
$L_1 \rightarrow L_2, id$	$L_2.type = L_1.type$ $insert_var(id.name, L_1.type)$
$L \rightarrow id$	$insert_var(id.name, L.type)$

A sequência de acções que um *parser bottom-up* executaria para a entrada “float p, q, r” pode ver-se a seguir:

Entrada	Símbolo	Regra
float p, q, r		
p, q, r	float	
p, q, r	T	$T \rightarrow float$
, q, r	T p	
, q, r	T L	$L \rightarrow id$
q, r	T L ,	
, r	T L , q	
, r	T L	$L \rightarrow L , id$
r	T L ,	
	T L , r	
	T L	$L \rightarrow L , id$
	D	$D \rightarrow T L$

Exemplo (cont.)

Note-se que sempre que se aplica uma regra gramatical relativa ao símbolo L , por baixo, na *stack* do *parser*, está sempre o símbolo T (e o seu atributo *type*). Logo a semântica pode ser simplificada, para a que se mostra em baixo:

Gramática	Acções semânticas
$D \rightarrow T L$	
$T \rightarrow \text{int}$	<code>type[top]=integer</code>
$T \rightarrow \text{float}$	<code>type[top]=float</code>
$L_1 \rightarrow L_2, \text{id}$	<code>insert_var(name[top], type[top-3])</code>
$L \rightarrow \text{id}$	<code>insert_var(name[top], type[top-1])</code>

Neste exemplo os atributos herdados *type* do não-terminal L vão buscar o seu valor ao atributo *type* de T . Este está na *stack* do *parser* sempre numa posição bem definida.